

SDC 학생강사

컴퓨터공학 입문자를 위한 자료구조론

3주차

CONTENTS

01

지난 시간 문제 분석

해당 목차에 대한 설명을 입력해 주세요

02

벡터

해당 목차에 대한 설명을 입력해 주세요

03

리스트와 시퀀스

해당 목차에 대한 설명을 입력해 주세요

01

지난 시간 문제 분석

문제 1

스택 성공



시간 제한	메모리 제한	제출	정답	맞힌 사람	정답 비율
0.5 초 (추가 시간 없음)	256 MB	315176	121439	87757	39.045%

문제

정수를 저장하는 스택을 구현한 다음, 입력으로 주어지는 명령을 처리하는 프로그램을 작성하시오.

명령은 총 다섯 가지이다.

- push X: 정수 X를 스택에 넣는 연산이다.
- pop: 스택에서 가장 위에 있는 정수를 빼고, 그 수를 출력한다. 만약 스택에 들어있는 정수가 없는 경우에는 -1을 출력한다.
- size: 스택에 들어있는 정수의 개수를 출력한다.
- empty: 스택이 비어있으면 1, 아니면 0을 출력한다.
- top: 스택의 가장 위에 있는 정수를 출력한다. 만약 스택에 들어있는 정수가 없는 경우에는 -1을 출력한다.

입력

첫째 줄에 주어지는 명령의 수 N ($1 \leq N \leq 10,000$)이 주어진다. 둘째 줄부터 N 개의 줄에는 명령이 하나씩 주어진다. 주어지는 정수는 1보다 크거나 같고, 100,000보다 작거나 같다. 문제에 나와있지 않은 명령이 주어지는 경우는 없다.

01

지난 시간 문제 분석

문제 2

큐 성공



시간 제한	메모리 제한	제출	정답	맞힌 사람	정답 비율
0.5 초 (추가 시간 없음)	256 MB	164830	79901	62886	49.923%

문제

정수를 저장하는 큐를 구현한 다음, 입력으로 주어지는 명령을 처리하는 프로그램을 작성하시오.

명령은 총 여섯 가지이다.

- push X: 정수 X를 큐에 넣는 연산이다.
- pop: 큐에서 가장 앞에 있는 정수를 빼고, 그 수를 출력한다. 만약 큐에 들어있는 정수가 없는 경우에는 -1을 출력한다.
- size: 큐에 들어있는 정수의 개수를 출력한다.
- empty: 큐가 비어있으면 1, 아니면 0을 출력한다.
- front: 큐의 가장 앞에 있는 정수를 출력한다. 만약 큐에 들어있는 정수가 없는 경우에는 -1을 출력한다.
- back: 큐의 가장 뒤에 있는 정수를 출력한다. 만약 큐에 들어있는 정수가 없는 경우에는 -1을 출력한다.

입력

첫째 줄에 주어지는 명령의 수 N ($1 \leq N \leq 10,000$)이 주어진다. 둘째 줄부터 N 개의 줄에는 명령이 하나씩 주어진다. 주어지는 정수는 1보다 크거나 같고, 100,000보다 작거나 같다. 문제에 나와있지 않은 명령이 주어지는 경우는 없다.

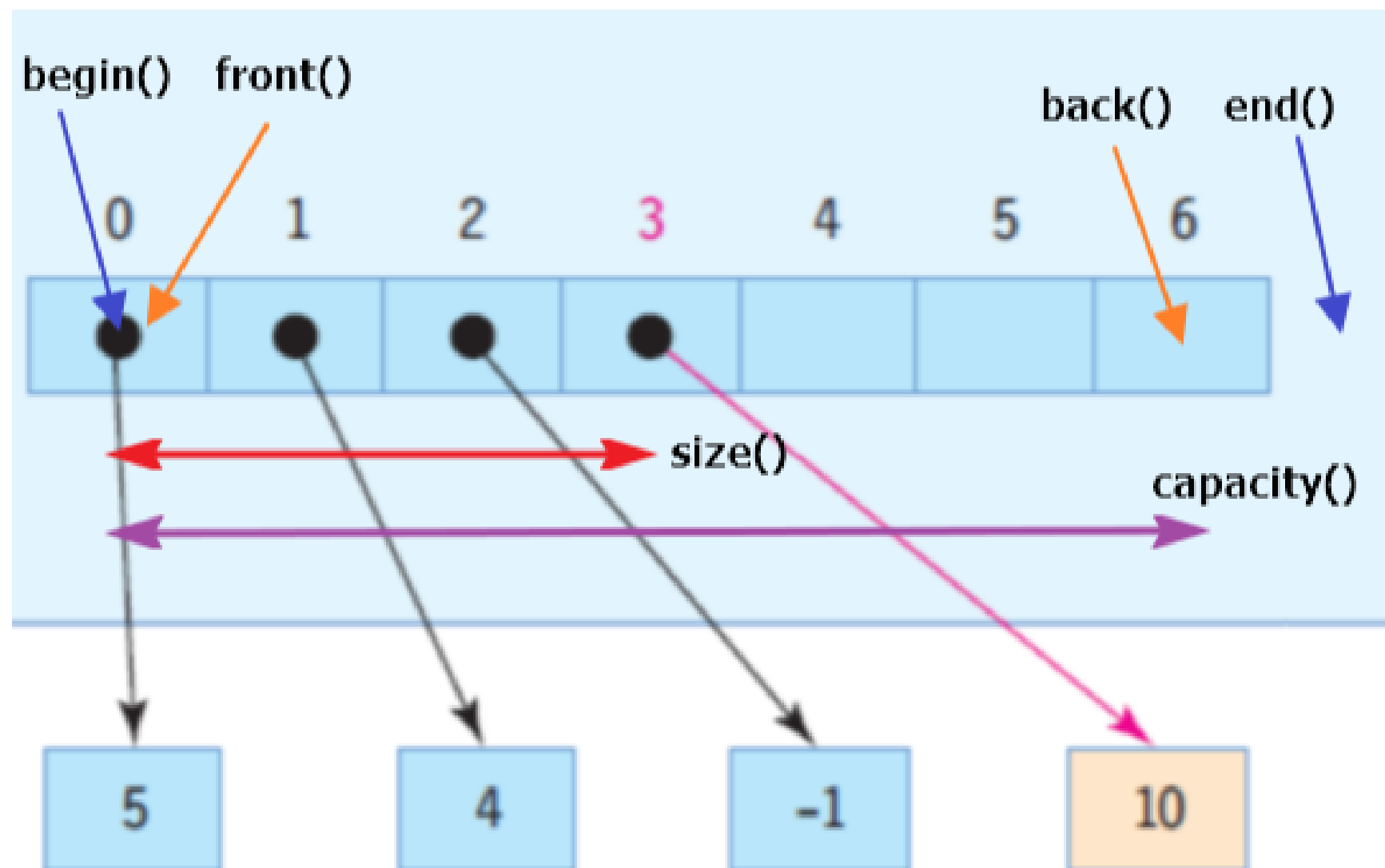
02

벡터

벡터 Vector

크기가 동적으로 변하는 동적배열로 필요에 따라 자동으로 크기를 늘리거나 줄일 수 있다.

동적 할당의 핵심은 컴파일 시점에 원소의 개수를 몰라도 된다는 것이다.



```
1 int *arr = new arr[size];  
2  
3 vector<int> arr  
4  
5
```

동적할당은 size가 정해지면 바꿀 수 없지만
vector는 자유롭게 증가, 감소가 가능하다

02

벡터

Vector ADT

at - i 번째 인덱스에 있는 원소값을 반환한다

set - i 번째 인덱스에 있는 원소값 x 로 바꾼다

insert - i 번째 인덱스에 새로운 값 x를 삽입한다

erase - i 번째 인덱스에 있는 값을 제거한다.

size

empty

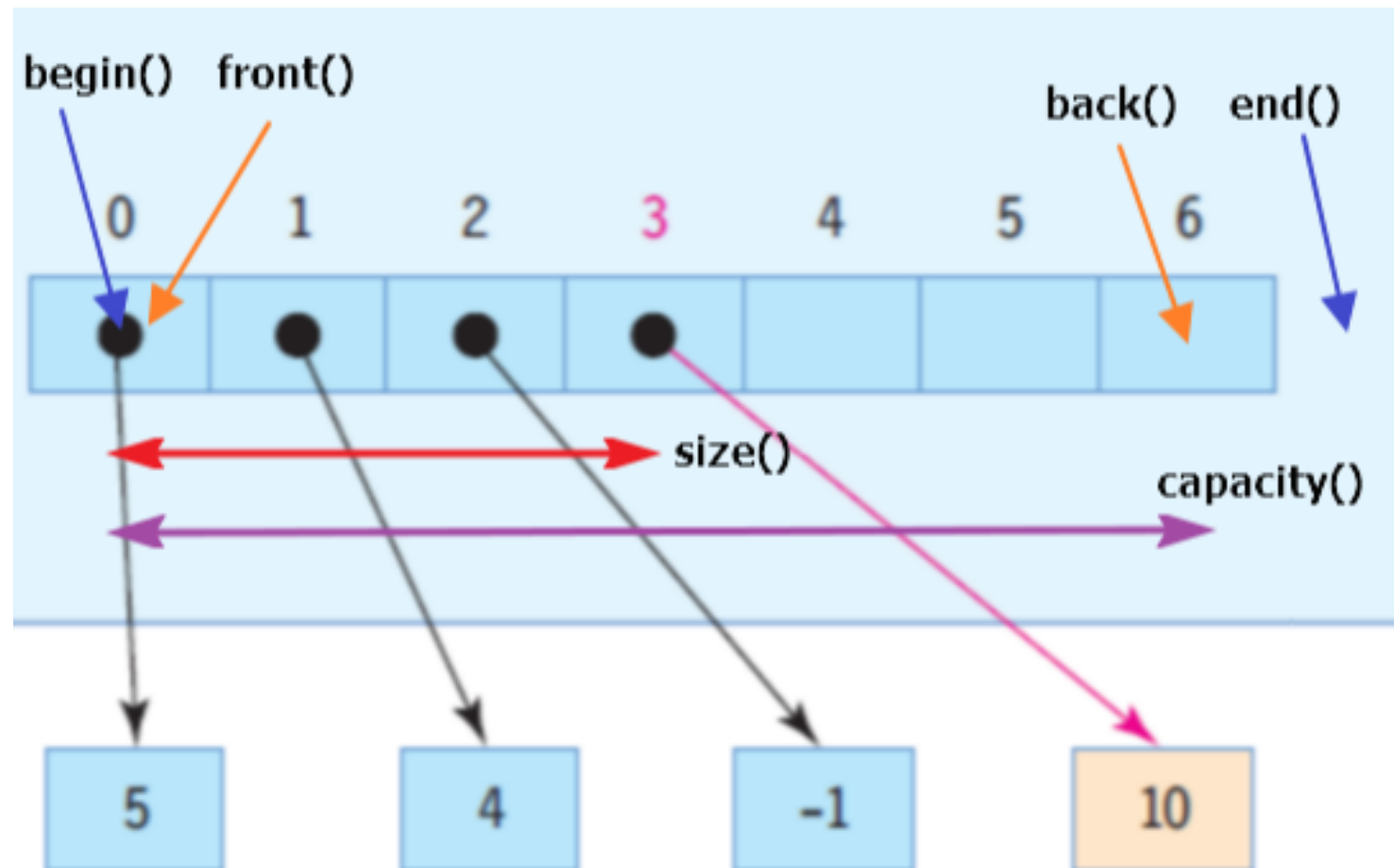
The Vector ADT

- The **Vector** or **Array List** ADT extends the notion of array by storing a sequence of objects
- An element can be accessed, inserted or removed by specifying its **index** (number of elements preceding it)
- An exception is thrown if an incorrect index is given (e.g., a negative index)
- Main methods:
 - **at**(integer i): returns the element at index i without removing it
 - **set**(integer i, object o): replace the element at index i with o
 - **insert**(integer i, object o): insert a new element o to have index i
 - **erase**(integer i): removes element at index i
- Additional methods:
 - **size**()
 - **empty**()

02

벡터

배열로 구현한 벡터



배열로 벡터를 구현하기 위해서는 size, capacity를 별도로 생각해서 관리해야 한다.

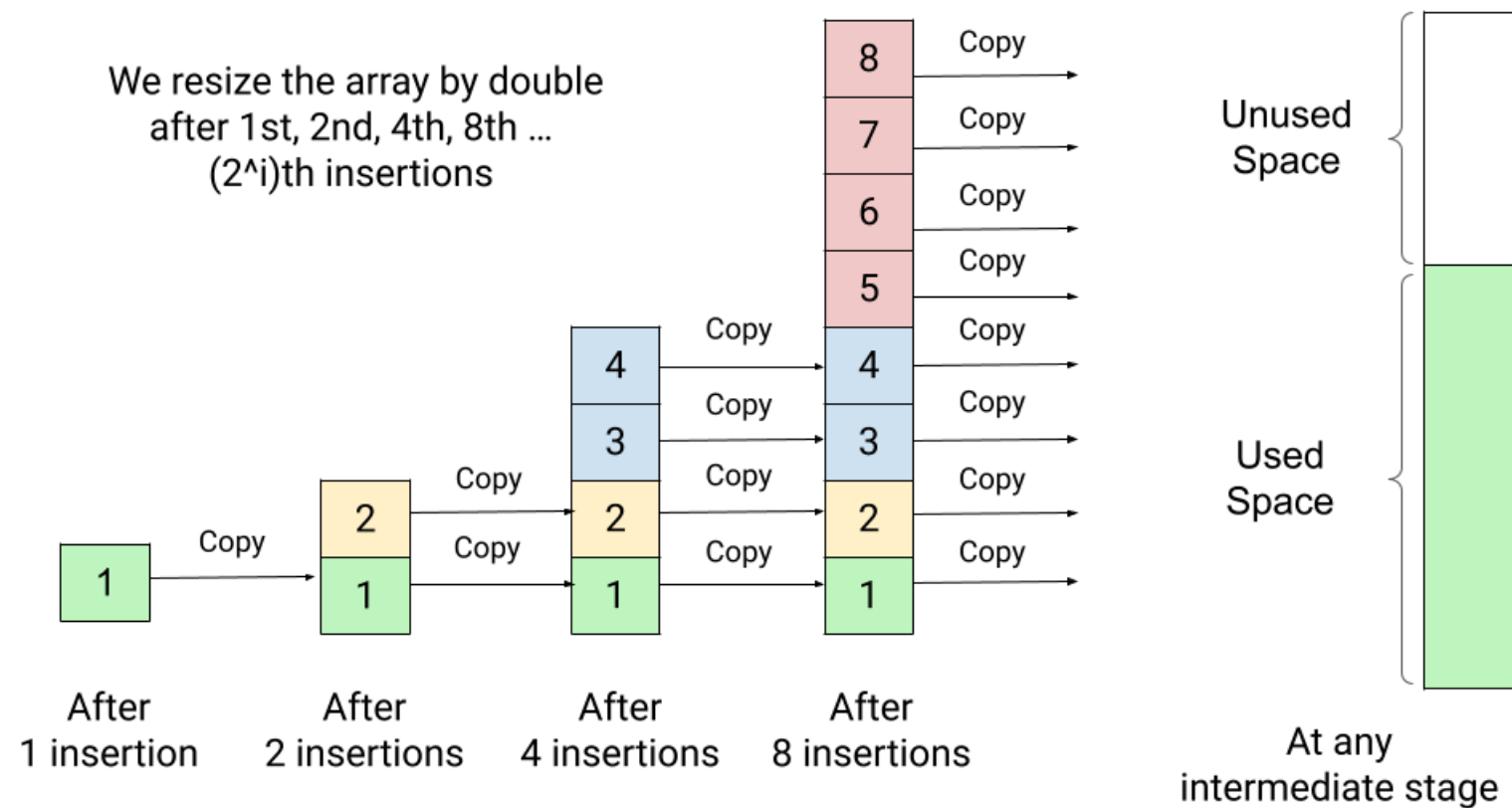
front, size, at, set 등의 연산은 배열로 구현하기 때문에 $O(1)$ 의 시간이 걸린다.

하지만 $\text{size} > \text{capacity}$ 의 경우 배열을 증가시키고 원래 데이터를 옮겨야 하기 때문에 추가적인 연산이 필요하다

02

벡터

배열로 구현한 벡터



가장 간단한 방법은 배열이 꽉 찼을때 원래 size의 2배 배열을 만들어서
원래 데이터를 복사 시키는 array doubling 기법을 활용

Implementing Stack with Array Doubling

- Array doubling is used behind the scenes to enlarge the array as necessary
- Assuming actual cost of push or pop is 1
 - when no enlarging of the array occurs
 - the actual cost of push is $1 + t \cdot n$
 - when array doubling is required
- Accounting scheme, assigning
 - accounting cost for a push to be $2t$
 - when no enlarging of array occurs
 - accounting cost for push to be $-t \cdot n + 2t$
 - when array doubling is required
- The amortized cost of each push operation is $1 + 2t$
- From the time the stack is created, the sum of the accounting cost must never be negative.

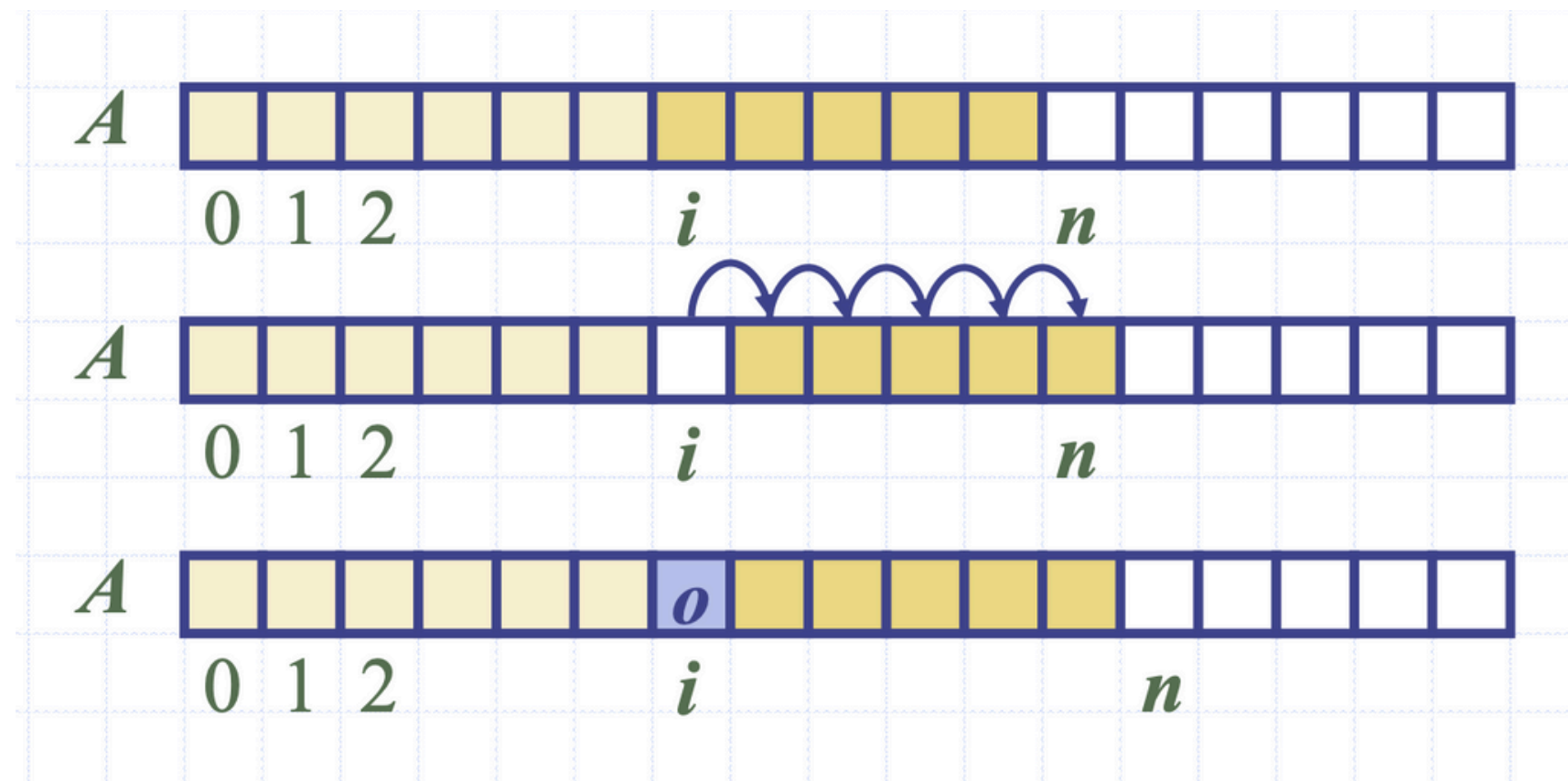
Cost	accounting cost
0	$2t$
t	$-2t + 2t$
$2t$	$-2t + 2t$
0	$2t$
$4t$	$-4t + 2t$
0	$2t$
0	$2t$
0	$2t$
$8t$	$-8t + 2t$
...	...

Handwritten notes:
 - "정당하게 감당함" (fairly bear)
 - "특정 연산이 발생하기 전까진" (before a specific operation occurs)
 - "일단적인 cost에 2t를 더해서 상환해준다." (repay by adding 2t to the amortized cost)

02

벡터

배열로 구현한 벡터 - Insert



Insert 연산을 구현할 때 내가 넣고자 하는 index 이후의 값을 뒤로 shift 해줘야 한다.

최악의 경우 모든 원소가 꽉 차있는 배열에
index = 0에 삽입을 하려 한다면
모든 원소를 한칸씩 뒤로 밀어야 한다 $O(n)$

02

벡터

배열로 구현한 벡터 - Insert

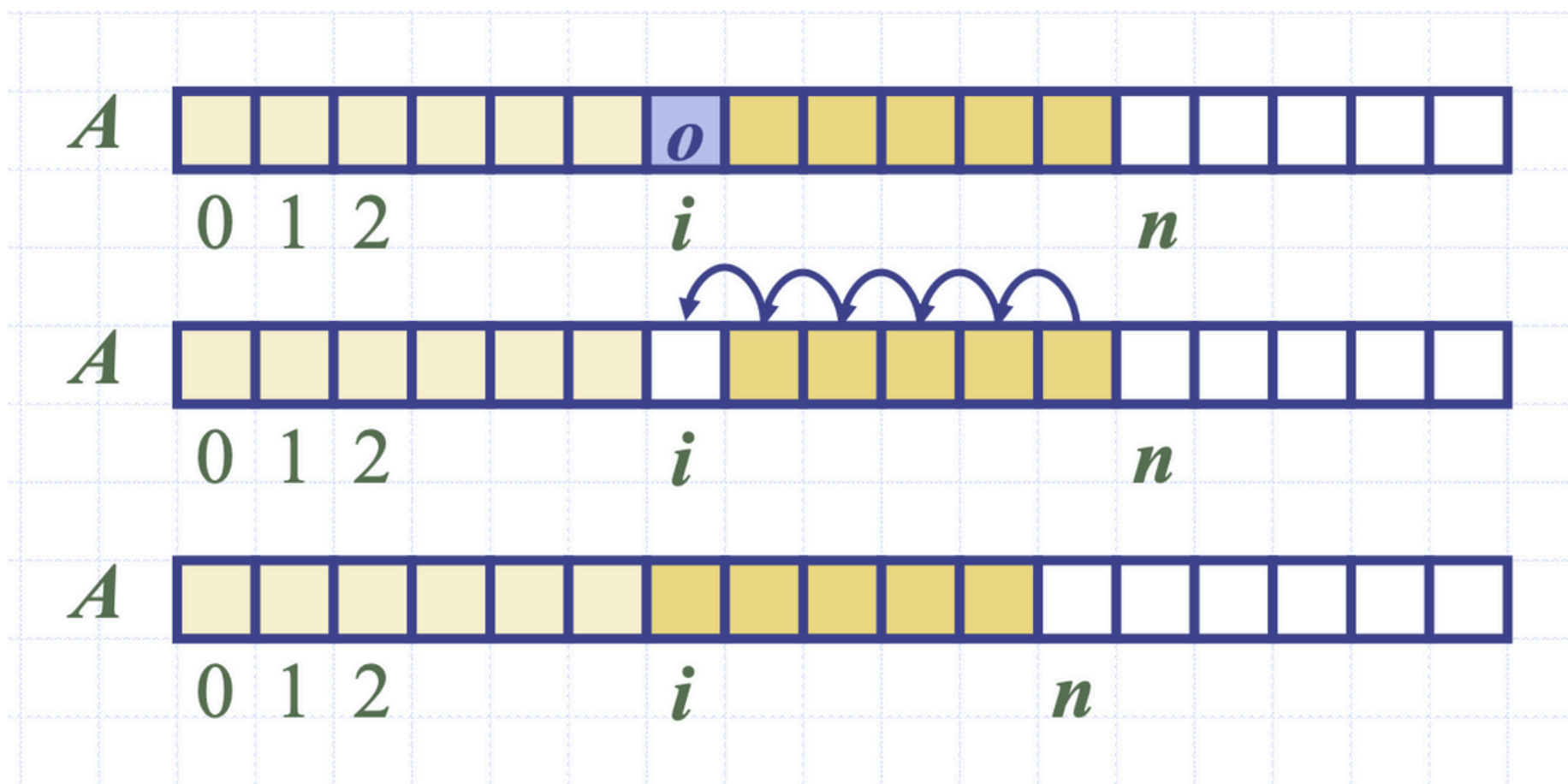
```
Algorithm insert(o)  
  if  $n = N$  then  
     $A \leftarrow$  new array of  
      size ...  
    for  $i \leftarrow 0$  to  $n-1$  do  
       $A[i] \leftarrow S[i]$   
    delete  $S$   
     $S \leftarrow A$   
     $N \leftarrow 2N$  {Doubling}  
   $S[n] \leftarrow o$   
   $n \leftarrow n + 1$ 
```

Insert 를 하는 순간 배열이 꽉 차있다면?
크기가 더 큰 배열에 새로 값을 할당시켜주는 로직이 필요하다

02

벡터

배열로 구현한 벡터 - Erase



erase 연산을 구현할때 내가 삭제 한 요소 이후의 값을 앞으로 가져오는 연산이 필요하다

최악의 경우 꼭 차 있는 벡터에서
index = 0 값을 erase 하는 경우
모든 값을 앞으로 한칸씩 땡겨줘야 한다 $O(n)$

02

벡터

Growable Array 벡터

배열 자체는 크기가 변할 수 없으니 더 큰 새로운 배열을 할당 받아 사용한다

Incremental strategy - 일정 크기만큼 증가시킴 eg) $N + c + c + c$

Doubling strategy - 기존 크기의 2배씩 할당함 eg) $N + 2N + 4N + 8N$

Incremental 전략의 경우 특정 c 마다 값을 증가시키는 연산이 이루어지기에 $O(n)$ 의 시간복잡도를 갖는다

직관적으로 생각했을때는 입력크기가 커질수록 doubling이 더 적은 빈도로 이루어지기에 더 효율적이다.
amotized한 방식으로 array doubling을 분석했을때 $O(1)$ 의 시간 복잡도를 갖기에 실제로 더 효율적임

02

벡터

링크드 리스트로 구현한 벡터

링크드 리스트로 구현했을때 배열과의 차이는 뭔지, 장단점이 뭐가 있는지 생각해보기

03

리스트와 시퀀스

Position ADT

데이터 구조내에서 단일 객체가 저장된 위치를 모델링 한다.
배열에서의 cell, 링크드 리스트에서 node와 같은 방식을 추상적으로 표현함

Position ADT

- The **Position** ADT models the notion of place within a data structure where a single object is stored
- It gives a unified view of diverse ways of storing data, such as
 - a cell of an array
 - a node of a linked list
- Just one method:
 - object **p.element()**: returns the element at position
 - In C++ it is convenient to implement this as ***p**

element - 자기 자신 객체의 위치를 반환함

03

리스트와 시퀀스

Node List ADT

Position들의 전후 관계, 열을 설정한다..

Node List ADT

- The **List** ADT models a sequence of positions storing arbitrary objects
- It establishes a before/after relation between positions
- Generic methods:
 - **size()**, **empty()**
- Iterators:
 - **begin()**, **end()**
- Update methods:
 - **insertFront(e)**, **insertBack(e)**
 - **removeFront()**, **removeBack()**
- Iterator-based update:
 - **insert(p, e)**
 - **remove(p)**

Iterator - 반복자

자료구조 내부 요소를 순회하는 객체로,
내부 구현을 몰라도 Iterator를 활용해 모든 요소에 접근이 가능하다

begin - 첫번째 요소를 가리키는 iterator

end - 마지막 요소 다음을 가리키는 iterator

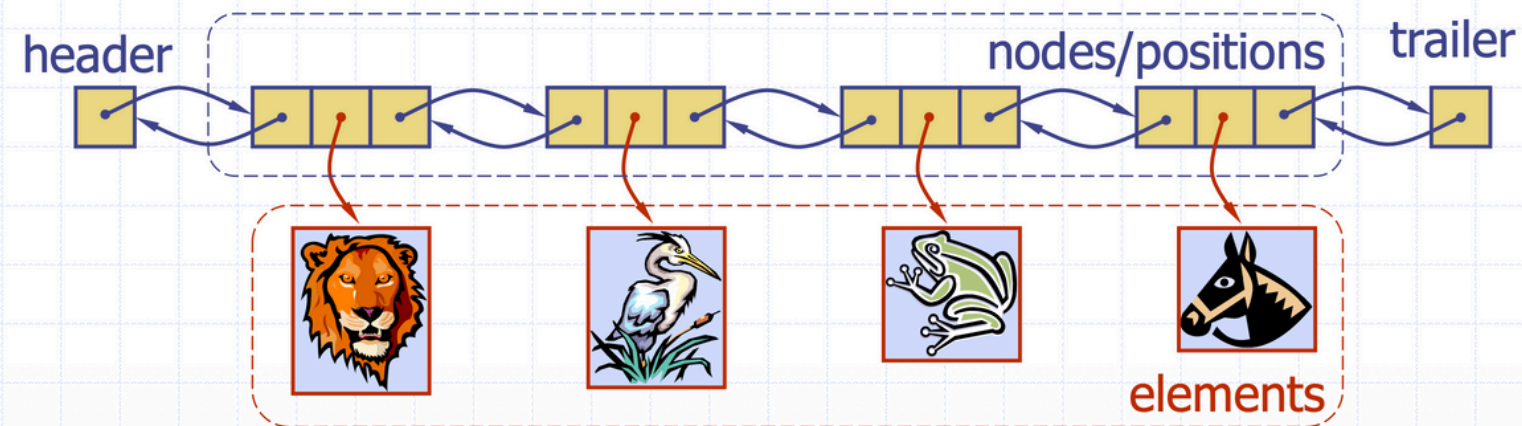
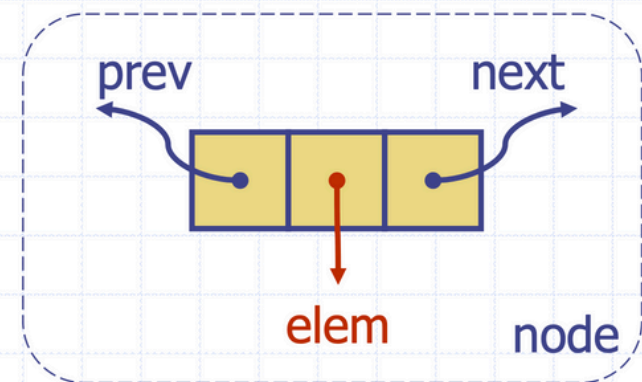
03

리스트와 시퀀스

List Position 객체를 데이터로 하는 자료구조로, 더블리 링크드 리스트로 구현된다.

Doubly Linked List

- A doubly linked list provides a natural implementation of the List ADT
- Nodes implement Position and store:
 - element
 - link to the previous node
 - link to the next node
- Special trailer and header nodes



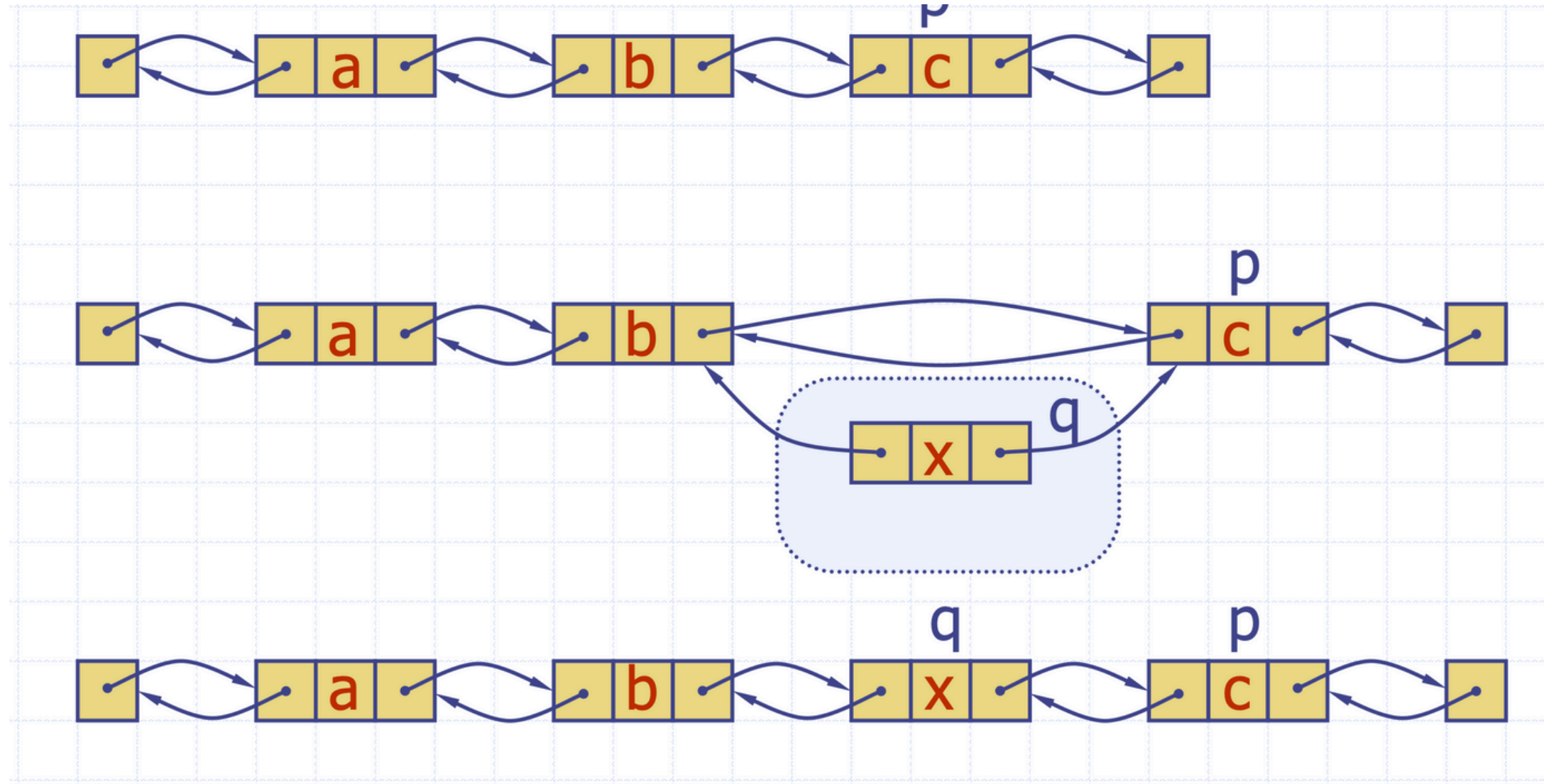
각 Node는 앞, 뒤 모두 연결되어 있어 참조가 가능하다.

Header, Trailer Node는 특수한 Node로 시작과 끝을 선언해준다

03

리스트와 시퀀스

List - Insert



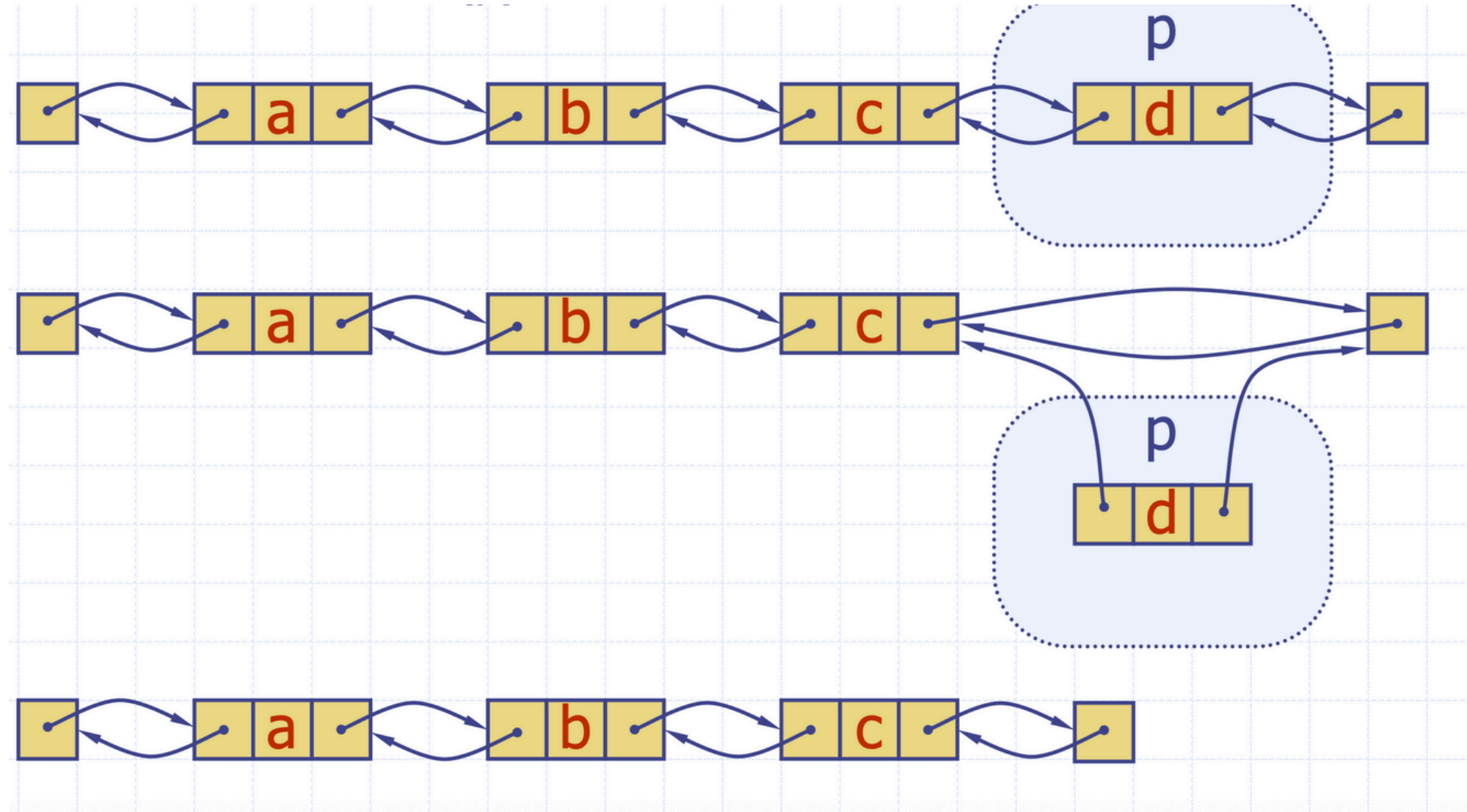
p 위치에 값을 삽입하기 위해서는

1. p의 위치를 알아내고
2. 새로 삽입하려는 노드의 이전노드, 다음 노드를 연결한다
3. 원래 있던 p위치의 연결을 끊는다

03

리스트와 시퀀스

List - remove



p 위치에 값을 제거 하기 위해서는

1. p의 위치를 알아내고
2. 제거하려는 노드 앞 뒤를 서로 연결한다.
3. 원래 있던 p위치의 연결을 끊는다

03

리스트와 시퀀스

시퀀스 벡터와 리스트의 접근 방법 (index, position)을 둘다 활용 가능하도록 만든 자료구조

Sequence ADT

- The **Sequence** ADT is the union of the Vector and List ADTs
- Elements accessed by
 - Rank, or
 - Position
- Generic methods:
 - **size()**, **isEmpty()**
- Vector-based methods:
 - **elemAtRank(r)**, **replaceAtRank(r, o)**, **insertAtRank(r, o)**, **removeAtRank(r)**
- List-based methods:
 - **first()**, **last()**, **before(p)**, **after(p)**, **replaceElement(p, o)**, **swapElements(p, q)**, **insertBefore(p, o)**, **insertAfter(p, o)**, **insertFirst(o)**, **insertLast(o)**, **remove(p)**
- Bridge methods:
 - **atRank(r)**, **rankOf(p)**

벡터 기반 접근

- index로 값을 조회, 수정, 제거

리스트 기반 접근

- 위치(Position)을 기반으로 값을 조회, 수정, 제거

Bridge method

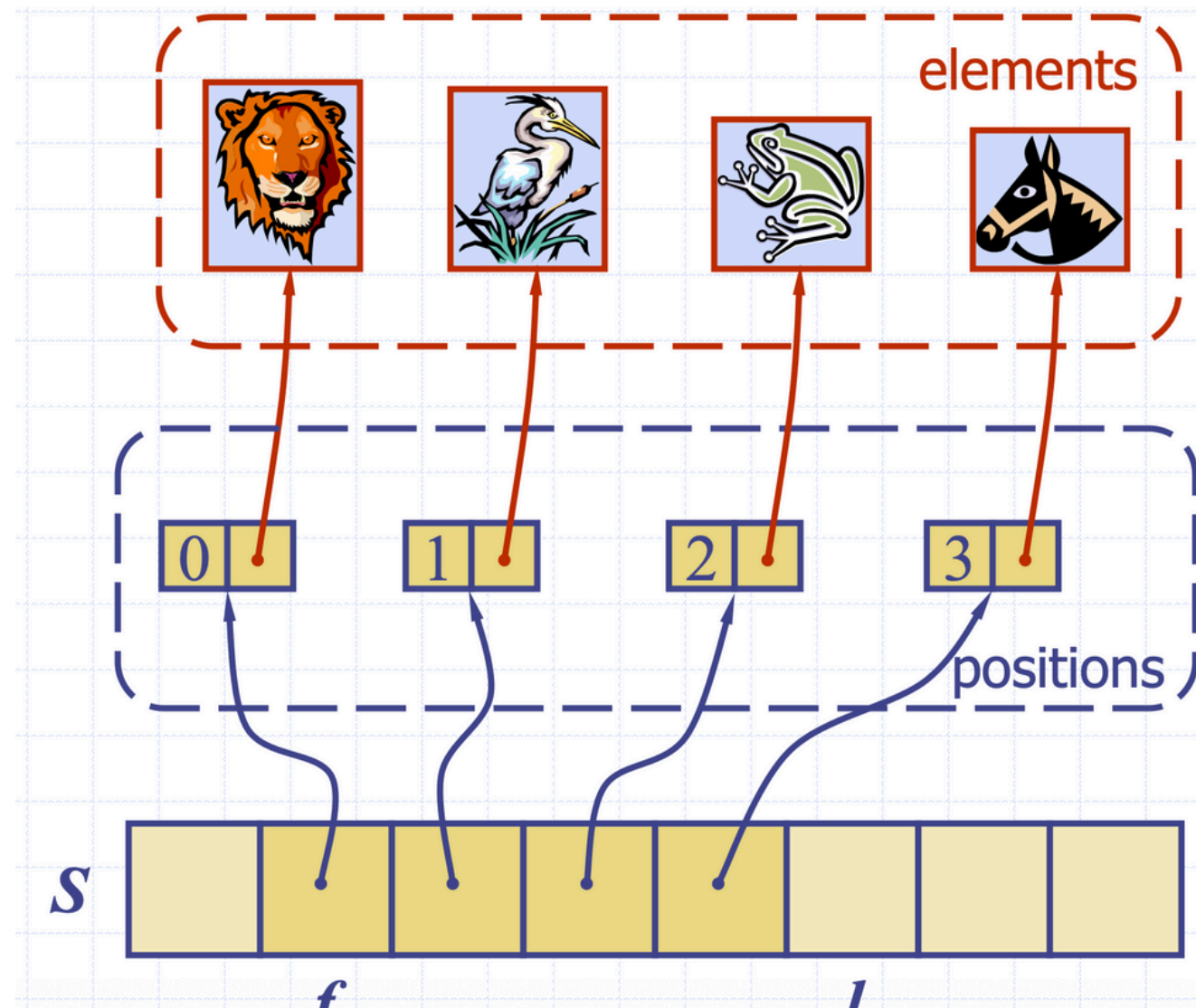
- index → position
- position → index로 변환하는 함수를 제공한다

시퀀스는 일반적으로 정렬된 값을 저장할때 많이 쓰인다.

03

리스트와 시퀀스

배열로 구현한 시퀀스



Position 을 저장하는배열을 선언

각 Positon에는 자신의 위치와 Element를 저장함

이때 배열은 주소값(Position에 대한 Pointer)만
저장하면 되기 때문에 각 4Byte의 저장공간만 필요하다

다만 erase, insert를 구현할때 index를 모두 수정 해줘야한다 $O(n)$

03

리스트와 시퀀스

리스트로 구현한 시퀀스

Node자체가 포인터로 연결됨

일반적 List구현과 동일, 하지만 index 계산을 위해 연산이 추가 존재함

Node 자체가 Position 역할을 해줌

```
header ↔ [🦁] ↔ [🦢] ↔ [🐙] ↔ [🐎] ↔ trailer  
prev | elem | next
```


03

리스트와 시퀀스

성능 비교

연산	배열 구현	연결리스트 구현	차이점
Vector 메서드			
<code>elemAtRank(r)</code>	$O(1)$ ✓	$O(n)$ ✗	배열은 인덱스 직접 접근
<code>insertAtRank(r, e)</code>	$O(n)$ ✗	$O(n)$ ✗	둘 다 느림
<code>removeAtRank(r)</code>	$O(n)$ ✗	$O(n)$ ✗	둘 다 느림
List 메서드			
<code>first()</code> / <code>last()</code>	$O(1)$ ✓	$O(1)$ ✓	둘 다 빠름
<code>after(p)</code> / <code>before(p)</code>	$O(1)$ ✓	$O(1)$ ✓	둘 다 빠름
<code>insertFirst(e)</code>	$O(n)$ ✗	$O(1)$ ✓	리스트가 훨씬 빠름!
<code>insertLast(e)</code>	$O(1)$ ✓	$O(1)$ ✓	둘 다 빠름
<code>insertAfter(p, e)</code>	$O(n)$ ✗	$O(1)$ ✓	리스트가 훨씬 빠름!
<code>remove(p)</code>	$O(n)$ ✗	$O(1)$ ✓	리스트가 훨씬 빠름!
기타			
메모리 효율	✓ 좋음	✗ 나쁨	포인터 오버헤드
캐시 지역성	✓ 좋음	✗ 나쁨	배열은 연속 메모리

03

리스트와 시퀀스

요세푸스 순열 요세푸스 문제

시간 제한	메모리 제한	제출	정답	맞힌 사람	정답 비율
2 초	256 MB	141854	72217	50559	49.601%

문제

요세푸스 문제는 다음과 같다.

1번부터 N번까지 N명의 사람이 원을 이루면서 앉아있고, 양의 정수 $K(≤ N)$ 가 주어진다. 이제 순서대로 K번째 사람을 제거한다. 한 사람이 제거되면 남은 사람들로 이루어진 원을 따라 이 과정을 계속해 나간다. 이 과정은 N명의 사람이 모두 제거될 때까지 계속된다. 원에서 사람들이 제거되는 순서를 (N, K)-요세푸스 순열이라고 한다. 예를 들어 (7, 3)-요세푸스 순열은 <3, 6, 2, 7, 5, 1, 4>이다.

N과 K가 주어지면 (N, K)-요세푸스 순열을 구하는 프로그램을 작성하시오.

입력

첫째 줄에 N과 K가 빈 칸을 사이에 두고 순서대로 주어진다. ($1 ≤ K ≤ N ≤ 5,000$)

04

과제

<https://www.acmicpc.net/problem/1406>